

pandas 데이터 재구조화(reshaping)

- 피벗팅(pivoting)
- 스택킹(stacking)과 언스택킹(unstacking)
- 멜팅(melting)과 와이드투롱(wide_to_long)
- 교차표(crosstab)
- explode

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}, Seaborn:{sns.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5, Seaborn:0.13.2

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

1. 피벗팅(pivoting)

예.

- 데이터1 : 제품이 생산될 때 마다 코드, 크기, 생산 수량을 기록

생산코드	지역	제품코드	크기	생산수량
1	경기도	BA3123	1 liter	5995
2	강원도	KA4121	2 liter	6645
3	강원도	DA1231	1 liter	18142
4	경기도	BA1231	2 liter	13891
5	전라도	FA1231	2 liter	19764
6	강원도	FA5121	2 liter	13593
7	경상도	NA0342	750 ml	17405
8	경상도	MA1241	2 liter	15876
9	경상도	DA1312	1 liter	10617
10	강원도	BA3123	750 ml	19574
11	강원도	KA4121	2 liter	21416
12	강원도	DA1231	2 liter	3810
13	경기도	BA1231	1 liter	9342
14	강원도	FA1231	750 ml	16544
15	강원도	FA5121	1 liter	24486
16	경상도	NA0342	2 liter	16172

- 피벗테이블1 : 지역과 제품코드별 생산수량 요약

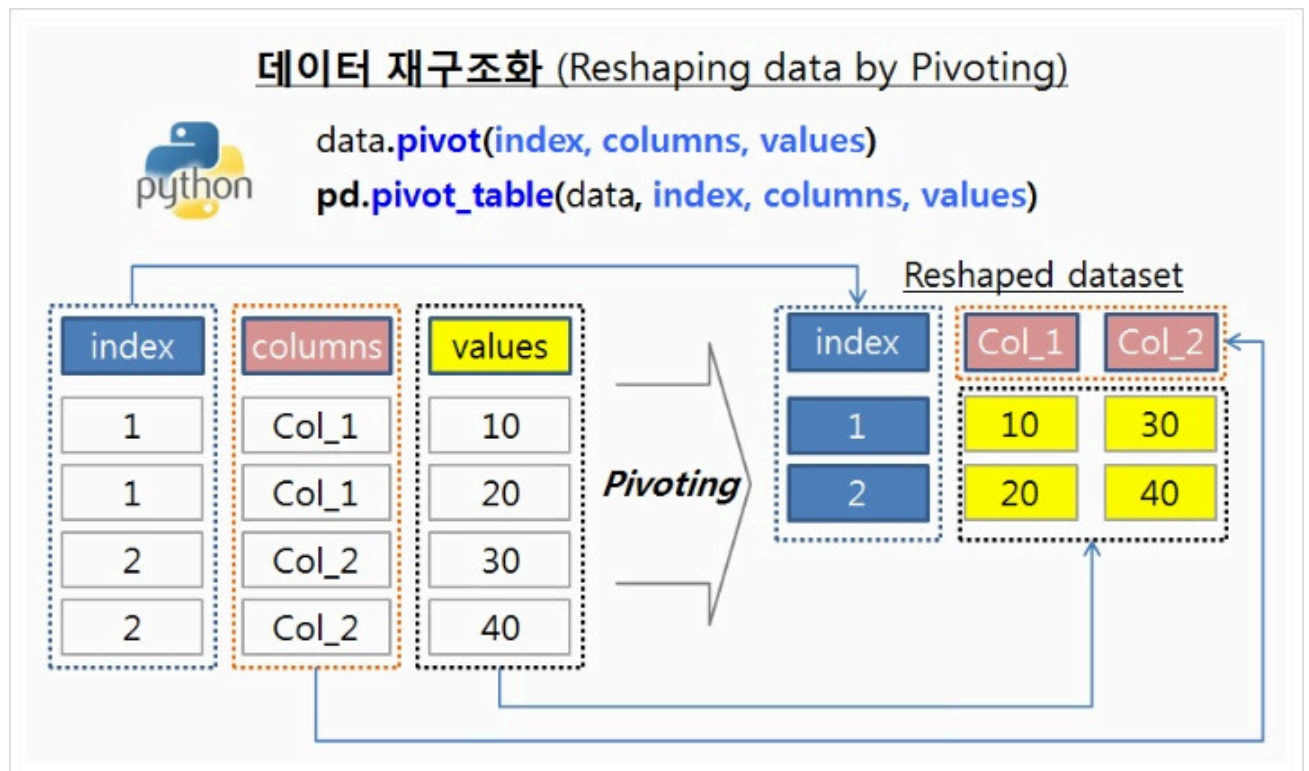
	BA1231	BA3123	CA0131	DA1231	DA1312	FA1231	FA5121	KA4121	MA1241	NA0342	총합계
강원도	8163	19574		71624	27572	28535	64779	40433	29041	37304	327025
경기도	32493	5995	18556		16354			20234	22865		116497
경상도		14871			10617	3636		25656	15876	46938	117594
전라도	14939	24903			11825	34405	28274				114346
총합계	55595	65343	18556	71624	66368	66576	93053	86323	67782	84242	7E+05

- 피벗테이블2 :제품크기와 제품코드별로 생산수량 요약

	BA1231	BA3123	CA0131	DA1231	DA1312	FA1231	FA5121	KA4121	MA1241	NA0342	총합계
1 liter	9342	20523		35976	22442	11991	65684		13444		179402
2 liter	36993	14871	18556	18553	35487	38041	27369	73951	31473	37412	332706
750 ml	9260	29949		17095	8439	16544		12372	22865	46830	163354
총합계	55595	65343	18556	71624	66368	66576	93053	86323	67782	84242	675462

Pivoting

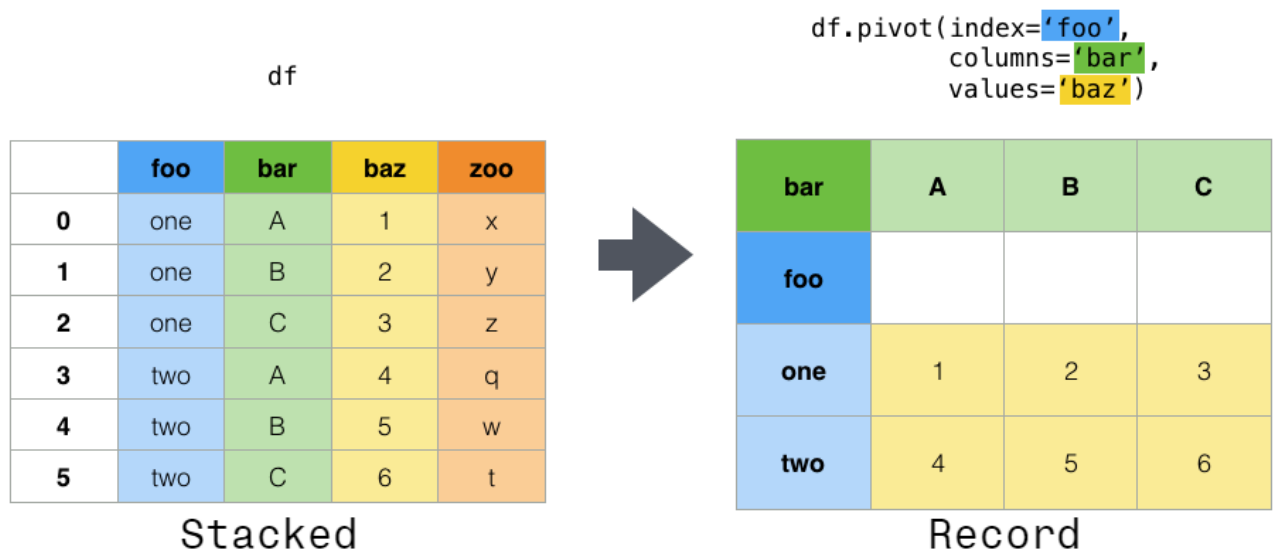
- 쌓여있는(stacked/long) 데이터를 원하는 형태로 가공하여(record/wide) 재구조화된 형식으로 보여주는 것
- stacked or long format -> record or wide format
 - stacked format : 각 주제(subject)에 대해 여러 행이 존재하는 형식
- 자료의 형태를 변경하기 위해 많이 사용하는 방법



- 출처 : <https://rfriend.tistory.com/275>

피버팅을 위한 메소드 : **pivot(), pivot_table()**

Pivot



pivot()

- `pandas.pivot(data, index=None, columns=None, values=None)`
- `DataFrame.pivot(index=None, columns=None, values=None)`
- <https://pandas.pydata.org/docs/reference/api/pandas.pivot.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot.html>

pivot_table()

- 방법 : 두 개의 키를 사용해서 데이터를 선택
- **`pivot_table(data, index=None, columns=None, values=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All', observed=False, sort=True)`**
 - `data` : 분석할 데이터 프레임의 메서드 형식일 때는 필요하지 않음
 - ex1. `pandas.pivot_table(df1, )`
 - ex2. `df1.pivot_table()`
 - `index` : 행 인덱스로 들어갈 키열 또는 키열의 리스트
 - `columns` : 열 인덱스로 들어갈 키열 또는 키열의 리스트
 - `values` : 분석할 데이터 프레임에서 분석할 열
 - `fill_value` : NaN이 호출될 때 대체값 지정
 - `margins` : 모든 데이터를 분석한 결과를 행으로 호출할 지 여부
 - `margins_name` : margins가 호출될 때 그 열(행)의 이름
- 피벗테이블을 작성할 때 반드시 설정해야 되는 인수
 - `data` : 사용 데이터 프레임
 - `index` : 행 인덱스로 사용할 필드(기준 필드로 작용됨)
 - `values` : 인덱스 명을 제외한 나머지 값(data)은 수치 data 만 사용함
 - `aggfunc`: 기본 함수가 평균(mean)함수 이기 때문에 각 데이터의 평균값이 반환

- https://pandas.pydata.org/docs/reference/api/pandas.pivot_table.html
- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot_table.html

예제1. pivot() 사용

```
In [3]: dates = ['2020-01-03', '2020-01-04', '2020-01-05']*4
dates
data = {'date': pd.to_datetime(dates),
        'value': range(1,13),
        'variable': ['A']*3 + ['B']*3 + ['C']*3 + ['D']*3
        }
df = pd.DataFrame(data)
df
```

```
Out[3]: ['2020-01-03',
         '2020-01-04',
         '2020-01-05',
         '2020-01-03',
         '2020-01-04',
         '2020-01-05',
         '2020-01-03',
         '2020-01-04',
         '2020-01-05',
         '2020-01-03',
         '2020-01-04',
         '2020-01-05']
```

```
Out[3]:
```

	date	value	variable
0	2020-01-03	1	A
1	2020-01-04	2	A
2	2020-01-05	3	A
3	2020-01-03	4	B
4	2020-01-04	5	B
5	2020-01-05	6	B
6	2020-01-03	7	C
7	2020-01-04	8	C
8	2020-01-05	9	C
9	2020-01-03	10	D
10	2020-01-04	11	D
11	2020-01-05	12	D

```
In [4]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   date        12 non-null    datetime64[us]
1   value       12 non-null    int64
2   variable    12 non-null    str
dtypes: datetime64[us](1), int64(1), str(1)
memory usage: 420.0 bytes
```

```
In [6]: df.pivot(index='date', columns='variable', values='value')
```

Out[6]:

	variable	A	B	C	D
	date				
	2020-01-03	1	4	7	10
	2020-01-04	2	5	8	11
	2020-01-05	3	6	9	12

```
In [7]: df['value2'] = df['value']*2
df
```

Out[7]:

	date	value	variable	value2
0	2020-01-03	1	A	2
1	2020-01-04	2	A	4
2	2020-01-05	3	A	6
3	2020-01-03	4	B	8
4	2020-01-04	5	B	10
5	2020-01-05	6	B	12
6	2020-01-03	7	C	14
7	2020-01-04	8	C	16
8	2020-01-05	9	C	18
9	2020-01-03	10	D	20
10	2020-01-04	11	D	22
11	2020-01-05	12	D	24

```
In [8]: result = df.pivot(index='date', columns='variable')
result
```

Out[8]:

		value				value2			
	variable	A	B	C	D	A	B	C	D
	date								
	2020-01-03	1	4	7	10	2	8	14	20
	2020-01-04	2	5	8	11	4	10	16	22
	2020-01-05	3	6	9	12	6	12	18	24

```
In [9]: result.columns
```

```
Out[9]: MultiIndex([( 'value', 'A'),
                    ( 'value', 'B'),
                    ( 'value', 'C'),
                    ( 'value', 'D'),
                    ('value2', 'A'),
                    ('value2', 'B'),
                    ('value2', 'C'),
                    ('value2', 'D')],
                  names=[None, 'variable'])
```

예제2. pivot() 사용

```
In [11]: data = {"도시": ["서울", "서울", "서울", "부산", "부산", "부산", "인천", "인천"],
                  "연도": ["2015", "2010", "2005", "2015", "2010", "2005", "2015", "2010"],
                  "인구": [9904312, 9631482, 9762546, 3448737, 3393191, 3512547, 2890451, 263203],
                  "지역": ["수도권", "수도권", "수도권", "경상권", "경상권", "경상권", "수도권", "수도권"]
                }
data
df2 = pd.DataFrame(data=data)
df2
```

```
Out[11]: {'도시': ['서울', '서울', '서울', '부산', '부산', '부산', '인천', '인천'],
          '연도': ['2015', '2010', '2005', '2015', '2010', '2005', '2015', '2010'],
          '인구': [9904312, 9631482, 9762546, 3448737, 3393191, 3512547, 2890451, 263203],
          '지역': ['수도권', '수도권', '수도권', '경상권', '경상권', '경상권', '수도권', '수도권']}
```

```
Out[11]:
```

	도시	연도	인구	지역
0	서울	2015	9904312	수도권
1	서울	2010	9631482	수도권
2	서울	2005	9762546	수도권
3	부산	2015	3448737	경상권
4	부산	2010	3393191	경상권
5	부산	2005	3512547	경상권
6	인천	2015	2890451	수도권
7	인천	2010	263203	수도권

- 각 도시의 연도별 인구 추이

```
In [12]: df2.pivot(index='도시', columns='연도', values='인구')
```

```
Out[12]:
```

	연도	2005	2010	2015
도시				
부산	3512547.0	3393191.0	3448737.0	
서울	9762546.0	9631482.0	9904312.0	
인천	NaN	263203.0	2890451.0	

- 지역별 도시에 대한 연도별 인구 추이

```
In [13]: df2.pivot(index=['지역', '도시'], columns='연도', values='인구')
```

Out[13]:

	연도	2005	2010	2015
--	----	------	------	------

지역 도시				
경상권	부산	3512547.0	3393191.0	3448737.0
수도권	서울	9762546.0	9631482.0	9904312.0
	인천	NaN	263203.0	2890451.0

=> pivot()함수는 long 형식의 데이터프레임을 wide 형식으로 변경하되 원데이터의 자리를 옮김

예제3.

```
In [15]: import datetime

df = pd.DataFrame({'A': 'one one two three'.split()*6,
                   'B': 'A B C'.split() * 8,
                   'C': 'foo foo foo bar bar bar'.split()*4,
                   'D': np.round(np.random.randn(24), 2),
                   'E': np.round(np.random.randn(24), 2),
                   'F': [datetime.datetime(2023, i, 1) for i in range(1,13)] +
                        [datetime.datetime(2023, i, 15) for i in range(1,13)]
                   })

df
df.info()
```

Out[15]:

	A	B	C	D	E	F
0	one	A	foo	-0.72	0.28	2023-01-01
1	one	B	foo	3.07	2.62	2023-02-01
2	two	C	foo	-1.29	-0.71	2023-03-01
3	three	A	bar	1.90	-0.36	2023-04-01
4	one	B	bar	0.54	-0.80	2023-05-01
5	one	C	bar	-0.38	-0.30	2023-06-01
6	two	A	foo	-0.11	-0.04	2023-07-01
7	three	B	foo	0.57	0.11	2023-08-01
8	one	C	foo	0.67	0.55	2023-09-01
9	one	A	bar	-0.35	1.49	2023-10-01
10	two	B	bar	0.55	0.41	2023-11-01
11	three	C	bar	-1.21	-1.01	2023-12-01
12	one	A	foo	0.75	-0.40	2023-01-15
13	one	B	foo	0.48	-0.80	2023-02-15
14	two	C	foo	-0.55	0.24	2023-03-15
15	three	A	bar	-1.24	0.75	2023-04-15
16	one	B	bar	0.91	0.81	2023-05-15
17	one	C	bar	0.68	0.56	2023-06-15
18	two	A	foo	-0.53	-0.21	2023-07-15
19	three	B	foo	0.49	0.88	2023-08-15
20	one	C	foo	-0.51	0.26	2023-09-15
21	one	A	bar	0.20	-0.45	2023-10-15
22	two	B	bar	-2.77	0.34	2023-11-15
23	three	C	bar	-0.15	-0.88	2023-12-15

```
<class 'pandas.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0    A      24 non-null      str
1    B      24 non-null      str
2    C      24 non-null      str
3    D      24 non-null      float64
4    E      24 non-null      float64
5    F      24 non-null      datetime64[us]
dtypes: datetime64[us](1), float64(2), str(3)
memory usage: 1.3 KB
```

- 피벗테이블 : pivot()함수 이용시

```
In [17]: # df.pivot(index=['A','B'], columns='C', values='D')
# ValueError: Index contains duplicate entries, cannot reshape
# 각 행과 열의 인덱스 조합이 유일(unique)하지 않아 pivot() 사용 불가
```


- 피벗테이블1 : `pivot_table()` 함수 사용

```
In [18]: df.pivot_table(index=['A','B'], columns='C', values='D')
# aggfunc='mean' 이 기본값으로 설정되어 있어 values의 값은 평균값이 나옴
```

```
Out[18]:
```

		C	bar	foo
	A	B		
one	A		-0.075	0.015
	B		0.725	1.775
	C		0.150	0.080
three	A		0.330	NaN
	B		NaN	0.530
	C		-0.680	NaN
two	A		NaN	-0.320
	B		-1.110	NaN
	C		NaN	-0.920

- 피벗테이블2 : `pivot_table(, aggfunc='sum')` 사용

```
In [19]: df.pivot_table(index=['A','B'], columns='C', values='D', aggfunc='sum')
```

```
Out[19]:
```

		C	bar	foo
	A	B		
one	A		-0.15	0.03
	B		1.45	3.55
	C		0.30	0.16
three	A		0.66	NaN
	B		NaN	1.06
	C		-1.36	NaN
two	A		NaN	-0.64
	B		-2.22	NaN
	C		NaN	-1.84

- 피벗테이블3 : `pivot_table(, aggfunc=['sum','mean'])` 사용

```
In [20]: df.pivot_table(index=['A','B'], columns='C', values='D',
                        aggfunc=['sum','mean'])
```

Out[20]:

		sum		mean	
	C	bar	foo	bar	foo
A	B				
one	A	-0.15	0.03	-0.075	0.015
	B	1.45	3.55	0.725	1.775
	C	0.30	0.16	0.150	0.080
three	A	0.66	NaN	0.330	NaN
	B	NaN	1.06	NaN	0.530
	C	-1.36	NaN	-0.680	NaN
two	A	NaN	-0.64	NaN	-0.320
	B	-2.22	NaN	-1.110	NaN
	C	NaN	-1.84	NaN	-0.920

- 피벗테이블4 : `pivot_table(, margins=True)` 사용

```
In [21]: df.pivot_table(index=['A','B'], columns='C', values='D',  
                        aggfunc=['sum','mean'], margins=True)
```

Out[21]:

		sum			mean		
	C	bar	foo	All	bar	foo	All
A	B						
one	A	-0.15	0.03	-0.12	-0.075	0.015000	-0.030000
	B	1.45	3.55	5.00	0.725	1.775000	1.250000
	C	0.30	0.16	0.46	0.150	0.080000	0.115000
three	A	0.66	NaN	0.66	0.330	NaN	0.330000
	B	NaN	1.06	1.06	NaN	0.530000	0.530000
	C	-1.36	NaN	-1.36	-0.680	NaN	-0.680000
two	A	NaN	-0.64	-0.64	NaN	-0.320000	-0.320000
	B	-2.22	NaN	-2.22	-1.110	NaN	-1.110000
	C	NaN	-1.84	-1.84	NaN	-0.920000	-0.920000
All		-1.32	2.32	1.00	-0.110	0.193333	0.041667

예제4. 타이타닉 데이터 : `pivot_table()` 사용

```
In [22]: titanic = sns.load_dataset('titanic')  
titanic.columns
```

```
Out[22]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
               'alive', 'alone'],  
              dtype='str')
```

```
In [23]: df = titanic[['age', 'sex', 'class', 'fare', 'survived']]
df.head()
df.info()
```

```
Out[23]:
```

	age	sex	class	fare	survived
0	22.0	male	Third	7.2500	0
1	38.0	female	First	71.2833	1
2	26.0	female	Third	7.9250	1
3	35.0	female	First	53.1000	1
4	35.0	male	Third	8.0500	0

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         714 non-null    float64
1   sex         891 non-null    str
2   class       891 non-null    category
3   fare        891 non-null    float64
4   survived    891 non-null    int64
dtypes: category(1), float64(2), int64(1), str(1)
memory usage: 29.0 KB
```

```
In [25]: df['class'].cat.categories
```

```
Out[25]: Index(['First', 'Second', 'Third'], dtype='str')
```

- 피벗테이블1 : 선실등급별 숙박객의 성별 평균 나이

```
In [26]: df.pivot_table(values='age', index='class', columns='sex', aggfunc='mean')
```

```
Out[26]:
```

	sex	female	male
class			
First		34.611765	41.281386
Second		28.722973	30.740707
Third		21.750000	26.507589

- 피벗테이블2 : 선실등급별 숙박객의 생존여부에 따른 평균 나이

```
In [27]: df.pivot_table(values='age', index='class', columns='survived', aggfunc='mean')
```

```
Out[27]:
```

	survived	0	1
class			
First		43.695312	35.368197
Second		33.544444	25.901566
Third		26.555556	20.646118

- 피벗테이블3 : 선실등급과 성별에 대해 생존여부에 따른 나이와 티켓값의 평균과 최대값 계산

```
In [28]: df.pivot_table(values=['age', 'fare'], index=['class', 'sex'],
                        columns='survived', aggfunc=['mean', 'max'])
```

Out[28]:

		mean				max			
		age		fare		age		fare	
		0	1	0	1	0	1	0	1
class	sex								
First	female	25.666667	34.939024	110.604167	105.978159	50.0	63.0	151.55	512.3292
	male	44.581967	36.248000	62.894910	74.637320	71.0	80.0	263.00	512.3292
Second	female	36.000000	28.080882	18.250000	22.288989	57.0	55.0	26.00	65.0000
	male	33.369048	16.022000	19.488965	21.095100	70.0	62.0	73.50	39.0000
Third	female	23.818182	19.329787	19.773093	12.464526	48.0	63.0	69.55	31.3875
	male	27.255814	22.274211	12.204469	15.579696	74.0	45.0	69.55	56.4958